

The purpose of this assignment is to create a lexical analyzer that takes HTML code and converts it into LaTeX formatting. It specifically handles comments, h1, h2, and h3 headers, preformatted text, paragraphs, and lists. Furthermore, it handles inline text editing of small, large, bold, italics, emphasized, strong, superscript, and subscript within paragraphs and lists.

The general structure of the project is to read HTML tags that take verbatim input, such as the “pre” tag or the “p” tag, with the exception of inline text tags within it. On top of paragraphs, this concept also applies to lists as well. There are six exclusive states in this program: “COMMENT”, “PARA”, “PRE”, “OL”, “UL”, and “LI”. These are written to tell Lex that when it enters into one of these states, it will only try the rules that apply to said state. This is used specifically for the inline text styles, such as making text bold or italicizing it.

Since there are two exclusive states that share the same concept, applying text formatting inside of another HTML block, these two were combined into one line being <PARA,LI>. This was done to reduce the amount of work needed, as separating them would’ve doubled the amount of rules unnecessarily.

Another important aspect to note is the “list_type” integer. Since a list item could be either an ordered list or unordered list, there was the need to implement a variable to track the state of a list in the code. The value of the type of list is passed to a “BEGIN” statement that says what kind of list it is once it reaches the end of a list item tag.

Debugging was done by running the code and checking for errors using the provided “test.html” file. The main issues were a few compile-time errors. This included trying to use “//” for comments instead of “/* ... */”, and including whitespace in the start condition tags, such as “<PARA, LI>” instead of “<PARA,LI>”. Once these errors were resolved, testing the actual conversion was done by copying the output of the .lex file and pasting it into a document on Overleaf. This was done to confirm more than just visually checking the code, but making sure a document actually compiled and displayed correctly.

The biggest issues faced involved getting inline formatting to work. It required refreshing my memory of how to apply start states, and then figuring out the logic of how to write the states to begin with. This was a major logic issue when figuring out how to break up statements into multiple sections and handle that. The initial versions had interpreted inline text literally, so it would display ... instead of applying the formatting to a given text. This is where figuring out how to apply exclusive states came to fruition. I also had to write out RegEx by hand to visualize my thinking. Whenever it was all laid out in that manner, it was easier to put it into code to create the lexical analyzer.